# Βιβλιογραφία

[1] “Interfaces (The Java™ Tutorials > Learning the Java Language > Interfaces and Inheritance).” https://docs.oracle.com/javase/tutorial/java/IandI/createinterface.html (accessed Oct. 27, 2021).

[2] “Multiple Inheritance of State, Implementation, and Type (The Java™ Tutorials > Learning the Java Language > Interfaces and Inheritance).” https://docs.oracle.com/javase/tutorial/java/IandI/multipleinheritance.html (accessed Oct. 27, 2021).

[3] “Περιοδικός πίνακας των χημικών στοιχείων,” Βικιπαίδεια. Oct. 23, 2021. Accessed: Nov. 02, 2021. [Online]. Available: https://el.wikipedia.org/w/index.php?title Περιοδικός_πίνακας_των_χημικών_στοιχείων&oldid=9105038

[4] “Minimax,” Wikipedia. Oct. 05, 2021. Accessed: Nov. 01, 2021. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Minimax&oldid=1048398851

[5] MIT OpenCourseWare, 6. Search: Games, Minimax, and Alpha-Beta, (Jan. 10, 2014). Accessed: Nov. 01, 2021. [Online Video]. Available: https://www.youtube.com/watch?v=STjW3eH0Cik

# Κεφάλαιο 15

**Σύνοψη**
*Στην ενότητα αυτή παρουσιάζεται η βασική διαχείριση των αρχείων. Πιο συγκεκριμένα, παρουσιάζεται η ανάγνωση δεδομένων με την Scanner, η ανάγνωση και εγγραφή σε δομημένα αρχεία, η ανάγνωση και εγγραφή σε αρχεία ψηφιολέξεων, η διαχείριση των αρχείων τυχαίας προσπέλασης, η σειριοποίηση και διάφορες χρήσιμες προκαθορισμένες λειτουργίες που σχετίζονται με αρχεία.*

**Προαπαιτούμενη γνώση**
*Η ενσωμάτωση, η κληρονομικότητα και ο πολυμορφισμός στο αντικειμενοστρεφές μοντέλο με Java, οι αφηρημένες κλάσεις καθώς και οι βασικές προκαθορισμένες κλάσεις, δηλαδή οι String, StringBuilder, ArrayList, LinkedList, Stack, PriorityQueue, HashSet, HashMap, HashTree, LocalDate, Localtime, Period και Duration. Οι διεπαφές στην Java.*

**Λέξεις κλειδιά**
*Είσοδος (input), έξοδος (output), τυχαία προσπέλαση (random access), σειριοποίηση (serialization)*

## 15 Διαχείριση Αρχείων

Η διαχείριση των αρχείων αποτελεί σημαντική δυνατότητα κάθε προγραμματιστικού περιβάλλοντος. Η Java παρέχει δύο πακέτα για τη διαχείριση των αρχείων. Το πακέτο java.io και το java.nio. Σε γενικές γραμμές, η λειτουργικότητα που προσφέρουν είναι περίπου η ίδια. Το πακετο java.io παρέχει τον καθιερωμένο τρόπο ανάγνωσης και εγγραφής σε αρχεία. Το πακέτο java.nio προστέθηκε στην 4η έκδοση της Java ως εναλλακτική λύση με στόχο να προσφέρει ταχύτερες λειτουργίες ανάγνωσης και εγγραφής. Η κύρια διαφορά μεταξύ των δύο πακέτων είναι πως το μεν java.io παρέχει συγχρονισμένες λειτουργίες (synchronous IO ή blocking IO) ανάγνωσης και εγγραφής, το δε java.nio παρέχει ασύγχρονες λειτουργίες (asynchronous IO ή non-blocking IO). Αυτό σημαίνει πως όταν ένα πρόγραμμα επιχειρήσει να διαβάσει ή να γράψει δεδομένα σε αρχείο, χρησιμοποιώντας το java.io, τότε η εκτέλεσή του σταματά έως ότου ολοκληρωθεί η λειτουργία ανάγνωσης ή εγγραφής. Όταν όμως χρησιμοποιείται το java.nio, τότε η εκτέλεση του προγράμματος συνεχίζεται παράλληλα με τη λειτουργία ανάγνωσης ή εγγραφής.

Στην ενότητα αυτή παρουσιάζονται βασικές κλάσεις του πακέτου java.io. Το πακέτο περιέχει σημαντικό αριθμό κλάσεων και διεπαφών. Εδώ συζητάμε ένα μέρος. Ωστόσο, οι κλάσεις και οι τεχνικές που παρουσιάζονται σε αυτήν την ενότητα μπορούν να αξιοποιηθούν ώστε να καλύψουν τις περισσότερες ανάγκες που προκύπτουν κατά την είσοδο και έξοδο των δεδομένων στις εφαρμογές.

Το πακέτο java.io βασίζεται στην έννοια των ροών εισόδου-εξόδου (Input/Output streams). Μια ροή είναι μια γραμμή επικοινωνίας μεταξύ του προγράμματος και μιας συσκευής εισόδου-εξόδου. Μέσα από τη ροή μεταφέρονται δεδομένα μεταξύ συσκευής και προγράμματος. Το πακέτο java.io υποστηρίζει τρεις βασικούς τύπους ροών:

- Ροές ψηφιολέξης (Byte Streams): Ροές μέσα από τις οποίες τα δεδομένα μεταφέρονται ψηφιολέξη προς ψηφιολέξη. Στο πακέτο java.io, όλες οι κλάσεις που υποστηρίζουν ροές ψηφιολέξεων είναι παράγωγες των αφηρημένων κλάσεων InputStream ή OutputStream.
- Ροές χαρακτήρων (Character Streams): Ροές μέσα από τις οποίες τα δεδομένα μεταφέρονται ως σειρά χαρακτήρων. Στο πακέτο java.io, όλες οι κλάσεις που υποστηρίζουν ροές χαρακτήρων είναι παράγωγες των αφηρημένων κλάσεων Reader ή Writer. Οι ροές χαρακτήρων χρησιμοποιούν τις ροές ψηφιολέξεων για να πραγματοποιήσουν είσοδο ή έξοδο αλλά υποστηρίζουν επιπλέον αυτόματη κωδικοποίηση χαρακτήρων (character encoding).
- Ροές προσωρινής αποθήκευσης (Buffered streams): Οι ροές προσωρινής αποθήκευσης αξιοποιούνται ως περιβλήματα (wrappers) για τις ροές ψηφιολέξεων και χαρακτήρων. Οι ροές ψηφιολέξεων και οι ροές χαρακτήρων διαβάζουν ή γράφουν μία ψηφιολέξη ή έναν χαρακτήρα κάθε φορά. Επομένως, οι λειτουργίες ανάγνωσης και εγγραφής είναι σημαντικά χρονοβόρες. Αντίθετα, αν ζητήσουμε από μία ροή προσωρινής αποθήκευσης να μας φέρει έναν χαρακτήρα από συσκευή εισόδου, η ροή θα διαβάσει έναν σημαντικό αριθμό χαρακτήρων, θα τους αποθηκεύσει στη μνήμη

και θα μας επιστρέψει τον χαρακτήρα που ζητήσαμε. Έτσι, στην επόμενη ανάγνωση δεν είναι απαραίτητη η πρόσβαση στη συσκευή εισόδου καθώς αρκετοί από τους επόμενους χαρακτήρες βρίσκονται ήδη στη μνήμη. Συμπερασματικά, οι ροές προσωρινής αποθήκευσης επιταχύνουν σημαντικά τις λειτουργίες ανάγνωσης και εγγραφής.

Πέραν των ανωτέρω, στην 5η έκδοση της Java προστέθηκε η κλάση Scanner που ανήκει στο πακέτο java.util την οποία έχουμε ήδη χρησιμοποιήσει για είσοδο από το πληκτρολόγιο και η οποία μπορεί να διασυνδεθεί και με άλλες συσκευές εισόδου. Γενικά, η κλάση χρησιμοποιείται εκτεταμένα και για αυτό η παρουσίαση ξεκινά με την Scanner.

## 15.1. Είσοδος με την Scanner

Με την Scanner έχουμε τη δυνατότητα να διαβάζουμε δεδομένα που αναπαριστούν θεμελιώδεις τύπους ή συμβολοσειρές [1]. Ένα αντικείμενο Scanner διασπά την είσοδό του σε επιμέρους τμήματα σύμφωνα με κάποιο διαχωριστικό (delimiter) που εξ ορισμού είναι ένας λευκός χαρακτήρας. Η ανάγνωση των δεδομένων γίνεται με τις ποικίλες μεθόδους hasNext και next που διαθέτει η Scanner, π.χ. hasNextDouble, nextDouble, hasNextInt, nextInt.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Locale;
import java.util.Scanner;

public class ScannerDemo {

    private static final String DOUBLEDATA =
"data"+File.separator+"doubledata.dat";

    public static void createDoubleData() throws IOException {
        PrintWriter out = new PrintWriter(DOUBLEDATA);
        for (double d = 0.5; d < 10; d += 0.4) {
            if (d >= 5 && d < 5.5) {
                out.println();
            }
            out.printf(Locale.US, "%5.1f", d);
        }
        out.close();
    }

    public static void readDoubles() throws FileNotFoundException {
        File file = new File(DOUBLEDATA);
        Scanner scanner = new Scanner(file);
        scanner.useLocale(Locale.US);
        while (scanner.hasNextDouble()) {
            System.out.println(scanner.nextDouble());
        }
        scanner.close();
    }

    public static void readLines() throws FileNotFoundException {
        File file = new File(DOUBLEDATA);
        Scanner scanner = new Scanner(file);
        scanner.useLocale(Locale.US);
        while (scanner.hasNextLine()) {
            System.out.println(scanner.nextLine());
        }
        scanner.close();
```

```
        }

        public static void main(String[] args) throws IOException {
            createDoubleData();
            readDoubles();
            readLines();
        }
    }
```

***Κώδικας 15.1*** *Επίδειξη ανάγνωσης με την Scanner.*

Σε αυτό το παράδειγμα, η ανάγνωση των δεδομένων γίνεται από ένα αρχείο με όνομα doubledata.dat που βρίσκεται στον φάκελο data. Ο δε φάκελος data είναι τοποθετημένος στον φάκελο εργασίας, δηλαδή στον γονικό φάκελο του project στον οποίο περιλαμβάνεται η κλάση ScannerDemo. Η σχετική αυτή διαδρομή αποθηκεύεται στη σταθερά DOUBLEDATA. Η σταθερά αυτή θα μπορούσε να οριστεί ως "data"+"\\"+"doubledata.dat", καθώς το διαχωριστικό στα ονόματα των διαδρομών αρχείων είναι ο χαρακτήρας "\". Αν επομένως διαμορφώσετε την DOUBLEDATA κατ' αυτόν τον τρόπο, η εφαρμογή σας θα τρέχει σωστά. Ωστόσο, η κωδικοποίηση αυτή δεν είναι ικανοποιητική για την Java που είναι διαπλατφορμική γλώσσα. Προτιμήστε τη διαμόρφωση που αξιοποιεί την File.separator και αφήστε την Java να επιλέξει το κατάλληλο διαχωριστικό ανάλογα με το λειτουργικό που θα τρέξει η εφαρμογή σας.
Το αρχείο doubledata.dat περιέχει τα ακόλουθα δεδομένα.

```
0.5  0.9  1.3  1.7  2.1  2.5  2.9  3.3  3.7  4.1  4.5  4.9
5.3  5.7  6.1  6.5  6.9  7.3  7.7  8.1  8.5  8.9  9.3  9.7
```

Πρόκειται για μια σειρά από πραγματικούς αριθμούς. Μπορείτε να δημιουργήσετε το αρχείο με κλήση της createDoubleData του κώδικα 15.1 ή χρησιμοποιώντας έναν συντάκτη κειμένου (text editor).

Η readDoubles διαβάζει τα δεδομένα του αρχείου αριθμό προς αριθμό. Μέσα στη συνάρτηση καταρχάς ορίζουμε ένα αντικείμενο της κλάσης File. Η κλάση File διευκολύνει στη διαχείριση των αρχείων. Μπορείτε να δείτε περισσότερα για την κλάση στην ενότητα 15.6. Στη συνέχεια, δημιουργούμε ένα αντικείμενο Scanner που συνδέεται με το συγκεκριμένο αρχείο. Η scanner.useLocale(Locale.US) έχει ως σκοπό να διασφαλίσει πως οι πραγματικοί αριθμοί θα μεταφραστούν από την Scanner θεωρώντας πως το διαχωριστικό των δεκαδικών είναι η τελεία και των χιλιάδων το κόμμα. Βέβαια, στο παράδειγμά μας δεν βλέπουμε το διαχωριστικό των χιλιάδων, καθώς τα δεδομένα μας είναι αριθμοί μικρότεροι του 10. Στη συνέχεια, η scanner ενόσω διαπιστώνει ύπαρξη πραγματικού αριθμού στη ροή εισόδου με την hasNextDouble, τον διαβάζει με την nextDouble. Κάθε είσοδος διαχωρίζεται από την επόμενη από την ύπαρξη ενός λευκού χαρακτήρα που στο παράδειγμά μας συνήθως είναι ένα διάστημα με εξαίρεση τον λευκό χαρακτήρα μεταξύ 4.9 και 5.3 που είναι ο χαρακτήρας newline.

Η readLines με παρόμοια τεχνική, διαβάζει από τη ροή εισόδου, μία γραμμή κάθε φορά, δηλαδή διαβάζει τα δεδομένα έως ότου συναντήσει τον χαρακτήρα newline. Κάθε γραμμή την επιστρέφει ως συμβολοσειρά. Οι ενδιάμεσοι λευκοί χαρακτήρες συμπεριλαμβάνονται στην επιστρεφόμενη συμβολοσειρά, όχι όμως και ο χαρακτήρας newline.

Προσέξτε πως οι συναρτήσεις του κώδικα 15.1 έχουν στη διεπαφή τους τη λέξη-κλειδί throws ακολουθούμενη από τύπους εξαιρέσεων. Αυτό οφείλεται σε πιθανά λάθη που μπορεί να προκύψουν κατά τη διαχείριση αρχείων. Γενικά σε αυτήν την ενότητα, τα πιθανά λάθη αντιμετωπίζονται με αυτόν τον τρόπο. Στην επόμενη ενότητα (κεφάλαιο 16) θα μιλήσουμε αναλυτικά για τις τεχνικές διαχείρισης των λαθών και θα δούμε περισσότερο σύνθετους και πιο αποτελεσματικούς τρόπους διαχείρισής τους.

Τέλος, προσέξτε πως κάθε συνάρτηση που αρχικοποιεί μία ροή την κλείνει κιόλας με την κλήση της close. Η ενέργεια αυτή είναι υποχρεωτική ώστε να διασφαλίζεται η ορθότητα των δεδομένων. Εξαίρεση αποτελεί η περίπτωση κατά την οποία η Scanner συνδέεται με την System.in όπως επισημάνθηκε στην ενότητα 4.3.

## 15.2 Δομημένα αρχεία δεδομένων

Πολύ συχνά, οι εφαρμογές μας διαχειρίζονται αρχεία στα οποία τα δεδομένα έχουν συγκεκριμένη δομή. Ας υποθέσουμε πως έχουμε μια σειρά από μαθητές των οποίων τα δεδομένα θέλουμε να αποθηκεύουμε σε ένα

αρχείο ή να τα διαβάζουμε από ένα αρχείο. Καταρχάς, θα πρέπει κάπως να αναπαριστούμε τα δεδομένα αυτά στη μνήμη του προγράμματός μας. Ο τυπικός τρόπος είναι ο σχεδιασμός κατάλληλης κλάσης. Έστω λοιπόν η κλάση Student, παράγωγη της Person.

```
import java.time.LocalDate;
import java.time.Month;
import java.util.Locale;

public class Student extends Person {

    private final double[] scores = new double[Subject.values().length];
    public final static int RECSIZE = 69;

    public Student(String id, String fName, String sName, LocalDate
birthday) {
        super(id, fName, sName, birthday);
    }

    public void setScore(Subject subject, double score) {
        scores[subject.ordinal()] = score;
    }

    public void setScores(double[] scores) {
        System.arraycopy(scores, 0, this.scores, 0, scores.length <
this.scores.length ? scores.length : this.scores.length);
    }

    public double getScore(Subject subject) {
        return scores[subject.ordinal()];
    }

    @Override
    public String toString() {
        return getId() + " " + getfName() + " " + getsName()
                + String.format(Locale.US, "%5.1f", getScore(Subject.MATH))
                + String.format(Locale.US, "%5.1f",
getScore(Subject.HISTORY))
                + String.format(Locale.US, "%5.1f",
getScore(Subject.LANGUAGE))
                + String.format(Locale.US, "%5.1f",
getScore(Subject.CHEMISTRY))
                + String.format(Locale.US, "%5.1f",
getScore(Subject.PHYSICS))
                + String.format(Locale.US, "%5.1f",
getScore(Subject.BIOLOGY))
                + " " + getBirthday();
    }

    public String toFixedWidth() {
        return String.format("%4s", getId())
                + String.format("%15s", getfName())
                + String.format("%15s", getsName())
                + String.format(Locale.US, "%4.1f", getScore(Subject.MATH))
                + String.format(Locale.US, "%4.1f",
getScore(Subject.HISTORY))
                + String.format(Locale.US, "%4.1f",
getScore(Subject.LANGUAGE))
                + String.format(Locale.US, "%4.1f",
getScore(Subject.CHEMISTRY))
                + String.format(Locale.US, "%4.1f",
getScore(Subject.PHYSICS))
```

```
                + String.format(Locale.US, "%4.1f",
    getScore(Subject.BIOLOGY))
                + getBirthday();
        }
    }
```

***Κώδικας 15.2*** *Η κλάση Student.*

Κάθε αντικείμενο Student εκτός από τα χαρακτηριστικά που κληρονομεί από την Person έχει και έναν πίνακα scores στον οποίο αποθηκεύονται οι βαθμοί του. Οι βαθμοί ελέγχονται με τη βοήθεια του απαριθμήσιμου τύπου Subject.

```
public enum Subject {
     MATH, HISTORY, LANGUAGE, CHEMISTRY, PHYSICS, BIOLOGY;
}
```

Ο δημιουργός της κλάσης καλεί απλώς τον δημιουργό της γονικής, ενώ οι μέθοδοι setScores, setScore και getScore χρησιμεύουν ως αναγνώστες και ρυθμιστές για τους βαθμούς του μαθητή.

Αν υποθέσουμε τώρα πως θέλουμε να γράψουμε ή να διαβάσουμε τα δεδομένα μιας ομάδας μαθητών, έχουμε δύο ειδών τεχνικές στη διάθεσή μας.

1. Τα δεδομένα ενός μαθητή διαχωρίζονται μεταξύ τους με κατάλληλο χαρακτήρα. Στην περίπτωσή μας, μπορούμε να χρησιμοποιήσουμε ένα διάστημα. Ωστόσο, αν η κλάση Student διέθετε μια μεταβλητή, π.χ. name στην οποία εκχωρούνταν τιμές που περιλαμβάνουν διάστημα, π.χ. “John Black”, τότε είναι προφανές πως θα έπρεπε να χρησιμοποιήσουμε άλλο χαρακτήρα διαχωρισμού. Τα αρχεία που διαμορφώνονται σύμφωνα με αυτήν την τεχνική ονομάζονται οριοθετημένα αρχεία (delimited files).
2. Κάθε πεδίο από τα δεδομένα ενός μαθητή εγγράφεται σε προκαθορισμένο πλάτος. Για παράδειγμα, κάθε βαθμός μπορεί να εγγραφεί σε πλάτος 4 χαρακτήρων καθώς οι βαθμοί μας παρακολουθούνται με ακρίβεια 1 δεκαδικού και δεν υπερβαίνουν τις 2 ακέραιες θέσεις. Μία θέση επιπλέον χρειάζεται για το διαχωριστικό των δεκαδικών, π.χ. 19.5. Το όνομα μπορεί να εγγραφεί σε πλάτος 15 χαρακτήρων εφόσον εκτιμούμε ότι στα δεδομένα μας δεν υπάρχει όνομα που να υπερβαίνει τους 15 χαρακτήρες. Η τεχνική αυτή ονομάζεται οριοθέτηση σταθερού πλάτους (fixed width).

Η συνάρτηση toString της κλάσης Student επιστρέφει μια String αναπαράσταση του αντικειμένου Student όπου κάθε επιμέρους στοιχείο διαχωρίζεται από το προηγούμενο με ένα διάστημα. Επομένως, ένα αντικείμενο Student μπορεί να γραφεί σε ροή εξόδου σύμφωνα με την πρώτη τεχνική οριοθέτησης χωρίς καμιά περαιτέρω επεξεργασία.

Η συνάρτηση toFixedWidth επιστρέφει μια String αναπαράσταση του Student με βάση την οριοθέτηση σταθερού πλάτους.

Ο προσδιοριστής διαμόρφωσης που λαμβάνει η String.format υπακούει στους ίδιους κανόνες με αυτόν που παρουσιάστηκε στην ενότητα 4.2.1 για την printf.

Στη συνέχεια, παρουσιάζεται πρώτα ένα παράδειγμα που γράφει και διαβάζει στοιχεία μαθητών σε οριοθετημένο αρχείο και ένα δεύτερο παράδειγμα που γράφει και διαβάζει σε αρχεία σταθερού πλάτους.

```
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;
import java.util.Scanner;

public class StudentsDelimited {

    static void createStudents(String fName) throws IOException {
```

```
        File file = new File(fName);
        BufferedWriter out = new BufferedWriter(new FileWriter(file));
        Student jim = new Student("113", "Jim", "Blue", LocalDate.of(2005,
Month.APRIL, 1));
        jim.setScores(new double[]{10, 11, 12, 13, 14, 15});
        Student jack = new Student("212", "Jack", "Green",
LocalDate.of(2005, Month.MAY, 1));
        jack.setScores(new double[]{11, 12, 13, 14, 15, 16});
        Student mary = new Student("111", "Mary", "Red", LocalDate.of(2005,
Month.JUNE, 1));
        mary.setScores(new double[]{12, 13, 14, 15, 16, 17});
        Student kely = new Student("201", "Kely", "Green",
LocalDate.of(2005, Month.JULY, 1));
        kely.setScores(new double[]{13, 14, 15, 16, 17, 18});
        out.write(jim + "\n");
        out.write(jack + "\n");
        out.write(mary + "\n");
        out.write(kely + "\n");

        out.close();
    }

    static ArrayList<Student> readStudents(String fName) throws
FileNotFoundException  {
        ArrayList<Student> rVal = new ArrayList<>();
        File input = new File(fName);
        Scanner scanner = new Scanner(input);
        while (scanner.hasNextLine()) {
            String st = scanner.nextLine();
            String[] stFields = st.split(" ");
            Student student = new Student(stFields[0], stFields[1],
stFields[2], LocalDate.parse(stFields[9]));
            student.setScore(Subject.MATH, Double.parseDouble(stFields[3]));
            student.setScore(Subject.HISTORY,
Double.parseDouble(stFields[4]));
            student.setScore(Subject.LANGUAGE,
Double.parseDouble(stFields[5]));
            student.setScore(Subject.CHEMISTRY,
Double.parseDouble(stFields[6]));
            student.setScore(Subject.PHYSICS,
Double.parseDouble(stFields[7]));
            student.setScore(Subject.BIOLOGY,
Double.parseDouble(stFields[8]));
            rVal.add(student);
        }
        scanner.close();
        return rVal;
    }

    public static void main(String[] args) throws IOException {
        String fName = "data" + File.separator + "studentsd.dat";
        createStudents(fName);
        ArrayList<Student> students = readStudents(fName);
        System.out.println(students);
    }
}
```

***Κώδικας 15.3*** *Ανάγνωση και εγγραφή σε οριοθετημένο αρχείο*

Στον κώδικα 15.3 η συνάρτηση createStudents δημιουργεί έναν αριθμό από αντικείμενα Students και τα αποθηκεύει σε αρχείο. Το αρχείο το διαχειριζόμαστε μέσω ενός FileWriter που είναι κλάση κατάλληλη για να

γράφουμε δεδομένα με τη μορφή χαρακτήρων. Επιπλέον χρησιμοποιείται ως περίβλημα ένα αντικείμενο BufferedWriter για επιτάχυνση της διαδικασίας εγγραφής. Κάθε μαθητής εισάγεται στο αρχείο ακολουθούμενος από τον χαρακτήρα newline. Μετά την εκτέλεση της createStudents, το αρχείο 'studentsd.dat' διαμορφώνεται ως ακολούθως:

```
113 Jim Blue 10.0 11.0 12.0 13.0 14.0 15.0 2005-04-01
212 Jack Green 11.0 12.0 13.0 14.0 15.0 16.0 2005-05-01
111 Mary Red 12.0 13.0 14.0 15.0 16.0 17.0 2005-06-01
201 Kely Green 13.0 14.0 15.0 16.0 17.0 18.0 2005-07-01
```

Η readStudents επιστρέφει ένα ArrayList από αντικείμενα Student που φορτώνει από το 'studentsd.dat'. Για να επιτύχει τη φόρτωση χρησιμοποιεί ένα αντικείμενο Scanner με το οποίο διαβάζει τις γραμμές του αρχείου μια προς μια. Η γραμμή εύκολα αποσυντίθεται στα επιμέρους στοιχεία της με την split. Στη συνέχεια, γνωρίζοντας τη γραμμογράφηση του αρχείου, μετατρέπει τα επιμέρους στοιχεία της γραμμής στους κατάλληλους τύπους και δημιουργεί το αντικείμενο Student που προσθέτει στο ArrayList που επιστρέφει. Τέλος, κλείνει τον Scanner. Στην main περιλαμβάνεται βασικός κώδικας ελέγχου.

```
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;
import java.util.Scanner;

public class StudentsToStdWidth {

    static void createStudents(String fName) throws IOException {
        File file = new File(fName);
        BufferedWriter out = new BufferedWriter(new FileWriter(file));
        Student jim = new Student("113", "Jim", "Blue", LocalDate.of(2005,
Month.MARCH, 1));
        jim.setScores(new double[]{10, 11, 12, 13, 14, 15});
        Student jack = new Student("212", "Jack", "Green",
LocalDate.of(2005, Month.MARCH, 2));
        jack.setScores(new double[]{11, 12, 13, 14, 15, 16});
        Student mary = new Student("911", "Mary", "Red", LocalDate.of(2005,
Month.MARCH, 3));
        mary.setScores(new double[]{12, 13, 14, 15, 16, 17});
        Student kely = new Student("201", "Kely", "Green",
LocalDate.of(2005, Month.MARCH, 4));
        kely.setScores(new double[]{13, 14, 15, 16, 17, 18});
        out.write(jim.toFixedWidth() + "\n");
        out.write(jack.toFixedWidth() + "\n");
        out.write(mary.toFixedWidth() + "\n");
        out.write(kely.toFixedWidth() + "\n");
        out.close();
    }

    public static Student toStudent(String st) {
        Student student = new Student(st.substring(0, 4).trim(),
                st.substring(4, 19).trim(),
                st.substring(19, 34).trim(),
                LocalDate.parse(st.substring(58))
        );
```

```
            student.setScore(Subject.MATH, Double.parseDouble(st.substring(34,
38).trim()));
            student.setScore(Subject.HISTORY,
Double.parseDouble(st.substring(38, 42).trim()));
            student.setScore(Subject.LANGUAGE,
Double.parseDouble(st.substring(42, 46).trim()));
            student.setScore(Subject.CHEMISTRY,
Double.parseDouble(st.substring(46, 50).trim()));
            student.setScore(Subject.PHYSICS,
Double.parseDouble(st.substring(50, 54).trim()));
            student.setScore(Subject.BIOLOGY,
Double.parseDouble(st.substring(54, 58).trim()));
            return student;
        }

        static ArrayList<Student> readStudents(String fName) throws
FileNotFoundException {
            ArrayList<Student> rVal = new ArrayList<>();
            File input = new File(fName);
            Scanner scanner = new Scanner(input);
            while (scanner.hasNextLine()) {
                String st = scanner.nextLine();
                Student student = toStudent(st);
                rVal.add(student);
            }
            scanner.close();
            return rVal;
        }

        public static void main(String[] args) throws IOException {
            createStudents("data" + File.separator + "students.dat");
            ArrayList<Student> students = readStudents("data" + File.separator
+ "students.dat");
            System.out.println(students);
        }
}
```

***Κώδικας 15.4*** *Ανάγνωση και εγγραφή σε αρχείο σταθερού πλάτους*

Στον κώδικα 15.4 επιδεικνύεται η ανάγνωση και εγγραφή σε αρχείο σταθερού πλάτους. Μετά από εκτίμηση του μέγιστου πιθανού πλάτους των δεδομένων ενός Student σχεδιάστηκε η ακόλουθη γραμμογράφηση του αρχείου. Το id τοποθετείται σε πλάτος 4 χαρακτήρων, το όνομα σε πλάτος 15, το επώνυμο σε πλάτος 15, οι βαθμοί σε πλάτος 4 με ακρίβεια 1 δεκαδικού. Τέλος, η ημερομηνία γέννησης γράφεται και διαβάζεται με την εξ ορισμού διαμόρφωσή της που καταλαμβάνει πλάτος 10 (yyyy-mm-dd). Η γραμμογράφηση αυτή επιστρέφεται από τη συνάρτηση toStandardWidth της κλάσης Student. Επομένως, η createStudents, το μόνο που κάνει είναι να δημιουργεί μια σειρά από αντικείμενα Student και να ενημερώνει το αρχείο students.dat. Η εγγραφή στο αρχείο γίνεται μέσω ενός FileWriter που είναι η κατάλληλη κλάση για εγγραφή χαρακτήρων σε αρχεία. Για επιτάχυνση της διαδικασίας εγγραφής χρησιμοποιείται ένα BufferedWriter ως περίβλημα του αντικειμένου FileWriter.
Μετά την εκτέλεση της createStudent, το αρχείο students.dat έχει ως εξής:

```
113            Jim            Blue10.011.012.013.014.015.02005-03-01
212           Jack           Green11.012.013.014.015.016.02005-03-02
911           Mary             Red12.013.014.015.016.017.02005-03-03
201           Kely           Green13.014.015.016.017.018.02005-03-04
```

Στη συνέχεια, η readStudents διαβάζει μία προς μία τις γραμμές του αρχείου και τις μετατρέπει σε αντικείμενα Student με τη βοήθεια της toStudent. Είναι προφανές πως η ανάγνωση τόσο οριοθετημένων αρχείων όσο και αρχείων σταθερού πλάτους προϋποθέτει τη γνώση της δομής τους. Στην main παρατίθεται βασικός κώδικας ελέγχου.

## 15.3 Ανάγνωση και εγγραφή σε αρχεία ψηφιολέξεων

Είδαμε πως μπορούμε να γράφουμε και να διαβάζουμε χαρακτήρες από ένα αρχείο. Μερικές φορές όμως τα αρχεία που θέλουμε να επεξεργαστούμε δεν είναι αρχεία κειμένου ώστε να περιέχουν χαρακτήρες. Για παράδειγμα, έστω ότι θέλουμε να αντιγράψουμε ένα αρχείο εικόνας. Σε αυτήν την περίπτωση θα πρέπει να διασυνδεθούμε μέσω ροής ψηφιολέξεων.

```
import java.io.BufferedInputStream;
import java.io.BufferedOutputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;

public class BinaryData {

    private static final String INPUT = "data" + File.separator +
"sunset.jpg";
    private static final String OUTPUT = "data" + File.separator +
"sunsetcopy.jpg";
    private static final String OUTPUTF = "data" + File.separator +
"sunsetcopyf.jpg";

    public static void copyBinary() throws FileNotFoundException,
IOException {
        FileInputStream in = new FileInputStream(INPUT);
        FileOutputStream out = new FileOutputStream(OUTPUT);
        int data;
        while (true) {
            data = in.read();
            if (data == -1) {
                break;
            } else {
                out.write(data);
            }
        }
        in.close();
        out.close();
    }

    public static void copyBinaryFast() throws FileNotFoundException,
IOException {
        BufferedInputStream in = new BufferedInputStream(new
FileInputStream(INPUT));
        BufferedOutputStream out = new BufferedOutputStream(new
FileOutputStream(OUTPUTF));
        int data;
        while (true) {
            data = in.read();
            if (data == -1) {
                break;
            } else {
                out.write(data);
            }
        }
        in.close();
        out.close();
    }

    public static void main(String[] args) throws IOException {
```

```
            copyBinary();
            copyBinaryFast();
        }
    }
```

***Κώδικας 15.5*** *Ανάγνωση και εγγραφή σε αρχεία ψηφιολέξεων*

Η κλάση BinaryData περιλαμβάνει δύο στατικές συναρτήσεις, την copyBinary και την copyBinaryFast. Και οι δύο αντιγράφουν την εικόνα sunset.jpg σε νέο αρχείο. Η copyBinary χρησιμοποιεί ένα αντικείμενο FileInputStream για ανάγνωση ψηφιολέξεων και ένα αντικείμενο FileOutputStream για εγγραφή ψηφιολέξεων. Η copyBinaryFast χρησιμοποιεί επιπλέον τα περιβλήματα BufferedInputStream και BufferedOutputStream. Δεδομένου ότι και οι δύο επιτελούν την ίδια εργασία, είναι εύκολη η εκτίμηση της επιτάχυνσης που προσφέρουν οι ροές προσωρινής μνήμης στις εφαρμογές. Τρέξτε τις δύο συναρτήσεις χωριστά και εκτιμήστε τη χρονική διαφορά στην εκτέλεσή τους.

Μετά το άνοιγμα της ροής, μέσα στην επαναληπτική διαδικασία, οι συναρτήσεις διαβάζουν μια προς μια τις ψηφιολέξεις. Η read διαβάζει την τρέχουσα ψηφιολέξη την οποία και επιστρέφει, οπότε στη συνέχεια η write την γράφει στο αρχείο εξόδου. Αν ωστόσο η read συναντήσει τέλος αρχείου, τότε επιστρέφει -1, οπότε διακόπτεται η επαναληπτική διαδικασία. Τέλος, οι συναρτήσεις κλείνουν τις ροές.

## 15.4 Αρχεία τυχαίας προσπέλασης

Ένας άλλος πολύ σημαντικός τρόπος διαχείρισης των αρχείων είναι η διαχείριση με τυχαία προσπέλαση (Random access). Η τυχαία προσπέλαση μας δίνει τη δυνατότητα να διαβάσουμε ή να γράψουμε άμεσα σε οποιοδήποτε σημείο του αρχείου θέλουμε. Ο χρόνος που απαιτείται για την προσπέλαση είναι ίδιος είτε προσπελαύνουμε την αρχή του αρχείου είτε οποιοδήποτε άλλο σημείο.

Η τυχαία προσπέλαση αξιοποιείται συνήθως σε δομημένα αρχεία. Ας πάρουμε για παράδειγμα το αρχείο student.dat που δημιουργήθηκε από τον κώδικα 15.4. Σε αυτό τα στοιχεία κάθε μαθητή καταλαμβάνουν μία γραμμή και συνιστούν μια εγγραφή (record). Όλες οι εγγραφές στο αρχείο έχουν το ίδιο πλάτος. Καθώς η συνάρτηση που παράγει αυτές τις εγγραφές, η toFixedWidth βρίσκεται στην κλάση Student, εκεί έχουμε τοποθετήσει και την πληροφορία σχετικά με το μήκος της εγγραφής. Πρόκειται για τη στατική μεταβλητή RECSIZE. Τα αρχεία τυχαίας προσπέλασης υποστηρίζουν τον δείκτη αρχείου (file pointer). Οι λειτουργίες ανάγνωσης και εγγραφής αρχίζουν από το σημείο που υποδεικνύει ο δείκτης αρχείου. Αν για παράδειγμα, θέλουμε να διαβάσουμε την πρώτη εγγραφή του αρχείου, ο δείκτης πρέπει να δείχνει στην ψηφιολέξη 0, αν θέλουμε να διαβάσουμε τη δεύτερη εγγραφή, ο δείκτης πρέπει να δείχνει στην ψηφιολέξη RECSIZE, αν θέλουμε την τρίτη εγγραφή, ο δείκτης πρέπει να δείχνει στην ψηφιολέξη 2*RECSIZE, κοκ.

Η κλάση που μας εξυπηρετεί για τυχαία προσπέλαση είναι η RandomAccessFile. Ο δείκτης του αρχείου τοποθετείται με τη μέδοθο seek. Επίσης, μπορούμε να μάθουμε πού δείχνει ο δείκτης με τη μέθοδο getFilePointer.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.io.RandomAccessFile;
import java.time.LocalDate;
import java.time.Month;

public class RandomAccess {

    public static Student read(RandomAccessFile f, int recNo) throws
IOException {
        int records = (int) (f.length() / Student.RECSIZE);
        if (recNo < 0 || recNo >= records) {
            throw new RuntimeException("reading past of file end");
        }
        long pos = recNo * Student.RECSIZE;
        f.seek(pos);
        String s = f.readLine();
```

```
            Student student = StudentsToStdWidth.toStudent(s);
            return student;
        }

        public static void update(RandomAccessFile f, int recNo, Student
    student) throws IOException {
            int records = (int) (f.length() / Student.RECSIZE);
            if (recNo < 0 || recNo >= records) {
                throw new RuntimeException("updating past of file end");
            }
            long pos = recNo * Student.RECSIZE;
            f.seek(pos);
            f.writeBytes(student.toFixedWidth());
        }

        public static void append(RandomAccessFile f, Student student) throws
    IOException {
            f.seek(f.length());
            f.writeBytes(student.toFixedWidth()+"\n");
        }

        public static void main(String[] args) throws FileNotFoundException,
    IOException {
            RandomAccessFile f = new
    RandomAccessFile("data"+File.separator+"students.dat", "rw");
            Student student = read(f, 3);
            System.out.println(student);
            student.setfName("Mike");
            student.setsName("Updater");
            update(f, 3, student);
            student=new Student("999", "Jane", "Appender",
    LocalDate.of(1999,Month.MARCH,20));
            student.setScores(new double[]{14,15,16,17,18,19});
            append(f,student);
            append(f,student);
            f.close();
        }
    }
```

***Κώδικας 15.6*** *Ανάγνωση και εγγραφή σε αρχεία με τυχαία προσπέλαση*

Η συνάρτηση read στον κώδικα 15.6 λαμβάνει ως παραμέτρους ένα αντικείμενο RandomAccessFile και έναν ακέραιο. Ο ακέραιος αναπαριστά τον αύξοντα αριθμό της εγγραφής μέσα στο αρχείο. Η συνάρτηση length της RandomAccessFile επιστρέφει το συνολικό μήκος του αρχείου σε ψηφιολέξεις. Το μήκος αυτό διαιρούμενο δια του μήκους της εγγραφής μας δίνει τον αριθμό των εγγραφών στο αρχείο. Έτσι, γίνεται έλεγχος μήπως ο αριθμός εγγραφής που ζητήθηκε από την read είναι εκτός των ορίων του αρχείου. Αν όχι, ο δείκτης τοποθετείται στη θέση recNo * Student.RECSIZE. Μετά την τοποθέτηση του δείκτη, διαβάζουμε μια εγγραφή και δημιουργούμε το αντικείμενο Student με την toStudent της StudentsToStdWidth.

Η συνάρτηση append μετακινεί τον δείκτη στο τέλος του αρχείου όπου προσθέτει μια νέα εγγραφή με τη βοήθεια της writeBytes και της toFixedWidth.

Η συνάρτηση update λαμβάνει ως παραμέτρους το αρχείο, τον αριθμό εγγραφής και ένα αντικείμενο Student. Αντικαθιστά τα στοιχεία της εγγραφής που βρίσκεται στη θέση recNo*Student.RECSIZE με τα στοιχεία του αντικειμένου Student. Προσέξτε πως στην writeBytes δεν προστίθεται χαρακτήρας newline όπως στην αντίστοιχη writeBytes της append, καθώς ο χαρακτήρας υπάρχει ήδη στο αρχείο.

## 15.5 Σειριοποίηση

Ας υποθέσουμε πως σε μια εφαρμογή-παιχνίδι θέλουμε να δώσουμε τη δυνατότητα στον χρήστη να διακόπτει το παιχνίδι και να το συνεχίζει ακριβώς από το σημείο στο οποίο διέκοψε. Στην περίπτωση αυτή θα μας ήταν